

Program - 2

Develop a Multi Stage Dockerfile for container orchestration

Project Structure:-

Program-2/

-->Dockerfile

-->package.json

-->package-lock.json

-->src./index.js

-->node_modules (created during npm install)

Step 1:Create a Dockerfile and add the content

```
mkdir Program-2
```

```
cd Program-2
```

```
nano Dockerfile
```

GNU nano 7.2

#Stage 1:build stage

FROM node:20-alpine AS builder

WORKDIR /app

COPY package.json package-lock.json ./

RUN npm install

COPY . .

RUN npm run build

#Stage 2:Production

FROM node:20-alpine

WORKDIR /app

COPY --from=builder /app/package.json ./

COPY --from=builder /app/package-lock.json ./

COPY --from=builder /app/dist ./dist

COPY --from=builder /app/node_modules ./node_modules

EXPOSE 8090

CMD ["node","dist/index.js"]

Step 2:Make a folder src and create index.js file

```
mkdir src
```

```
nano index.js
```

1rv24mc085_rohit@rohit-Victus-by-HP-Gaming-Laptop-15-fa1xxx: ~/my-docker-... ×

GNU nano 7.2

```
const express=require('express');
const app=express();
const PORT=8090;

app.get('/',(req,res)=> {
res.send('Hello from multi stage docker');
});

app.listen(PORT,()=> {
console.log('server is running on port ${PORT}');
});
```

Step 3:Create a package.json file

nano package.json

1rv24mc085_rohit@rohit-Victus-by-HP-Gaming-Laptop-15-fa1xxx: ~/my-docker-... ×

GNU nano 7.2

```
{
  "name": "program-2",
  "version": "1.0.0",
  "description": "A Node.js app with multi_stage",
  "main": "index.js",
  "scripts": {
    "start": "node dist/index.js",
    "build": "mkdir -p dist && cp -r src/* dist/"
  },
  "author": "Rohit",
  "license": "MIT",
  "dependencies": {
    "express": "^4.21.2"
  }
}
```

Step 4: Install node modules

```
npm install
```

Step 4: Execute docker build command

```
sudo docker build -t program-2 .
```

```

root@3D: ~/Program-2S docker build -t program-2 .
[*] Building 1.7s (15/15) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 467B
=> [internal] load metadata for docker.io/library/node:20-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [builder 1/6] FROM docker.io/library/node:20-alpine@sha256:6178e78b972f79c335df281f4b7674a2d85071aae2af020ffa39f0a770265435
=> [internal] load build context
=> => transferring context: 42.14kB
=> CACHED [builder 2/6] WORKDIR /app
=> CACHED [builder 3/6] COPY package.json package-lock.json ./
=> CACHED [builder 4/6] RUN npm install
=> [builder 5/6] COPY . .
=> [builder 6/6] RUN npm run build
=> CACHED [stage-1 3/6] COPY --from=builder /app/package.json ./
=> CACHED [stage-1 4/6] COPY --from=builder /app/package-lock.json ./
=> [stage-1 5/6] COPY --from=builder /app/dist ./dist
=> [stage-1 6/6] COPY --from=builder /app/node_modules ./node_modules
=> exporting to image
=> => exporting layers
=> => writing image sha256:2a2ff1c69d4685d2d93a8be3c8132f1da8fbd9a922ac29f88ee72730c9894221
=> => naming to docker.io/library/program-2

```

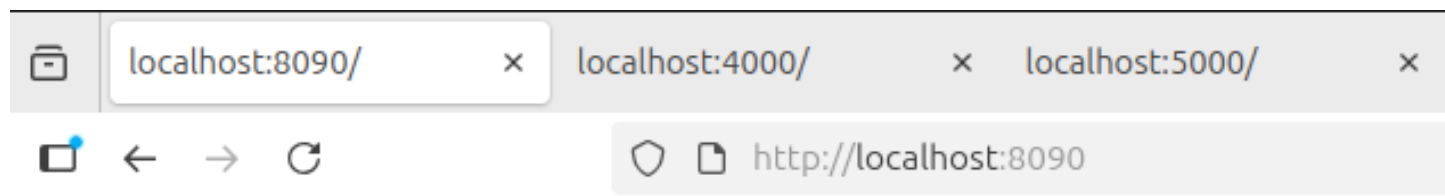
Step 5:Execute docker run command

```
sudo docker run -d -p 8090:8090 program-2
```

```
Rohit@JD:~/Program-2$ docker run -d -p 8090:8090 program-2
99f889b340a3b0bae29f320f6e836c42ef57a6ef7d439547b66c54e89bf247ec
```

Step 6:Test the container in the browser through localhost

http://localhost:8090



Hello from multi stage docker